CS 0007: Introduction to Java Lecture 2

Nathan Ong
University of Pittsburgh
September 6, 2016

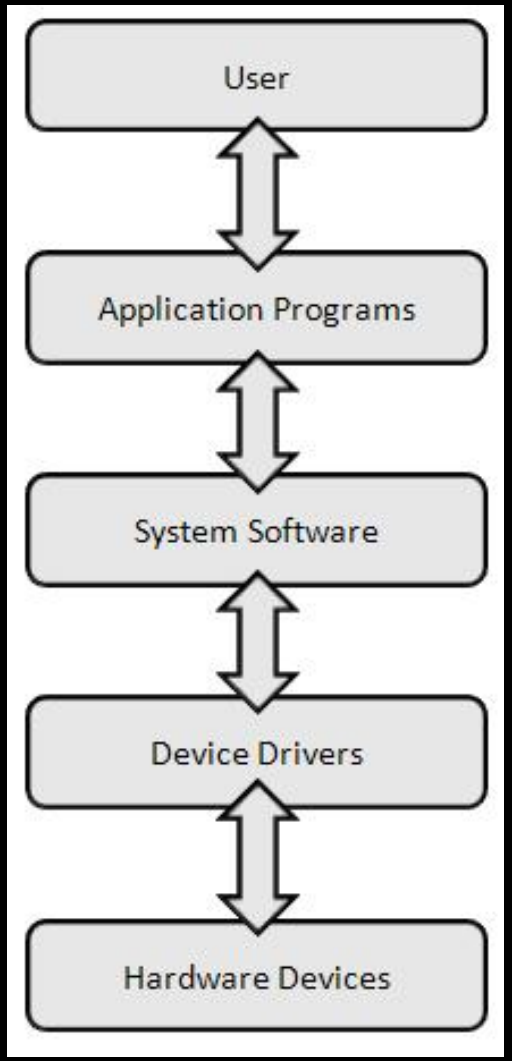
Outline

- Recap of last lecture
- Java’s “Place of Residence”
- Your First Program

Why Learn Java?

- Widely used
- Constantly updating and receiving support
- Easily understood due to standardized and consistent syntax
- Forces code readability
- Virtual Machine isolates running code

Software Layers



- Applications: Software that interfaces with the user
- System Software: Operating System (e.g. Windows, Mac OSX, Linux)
- Device Drivers: Software to talk to non-computer devices (e.g. printers, webcams, etc.)

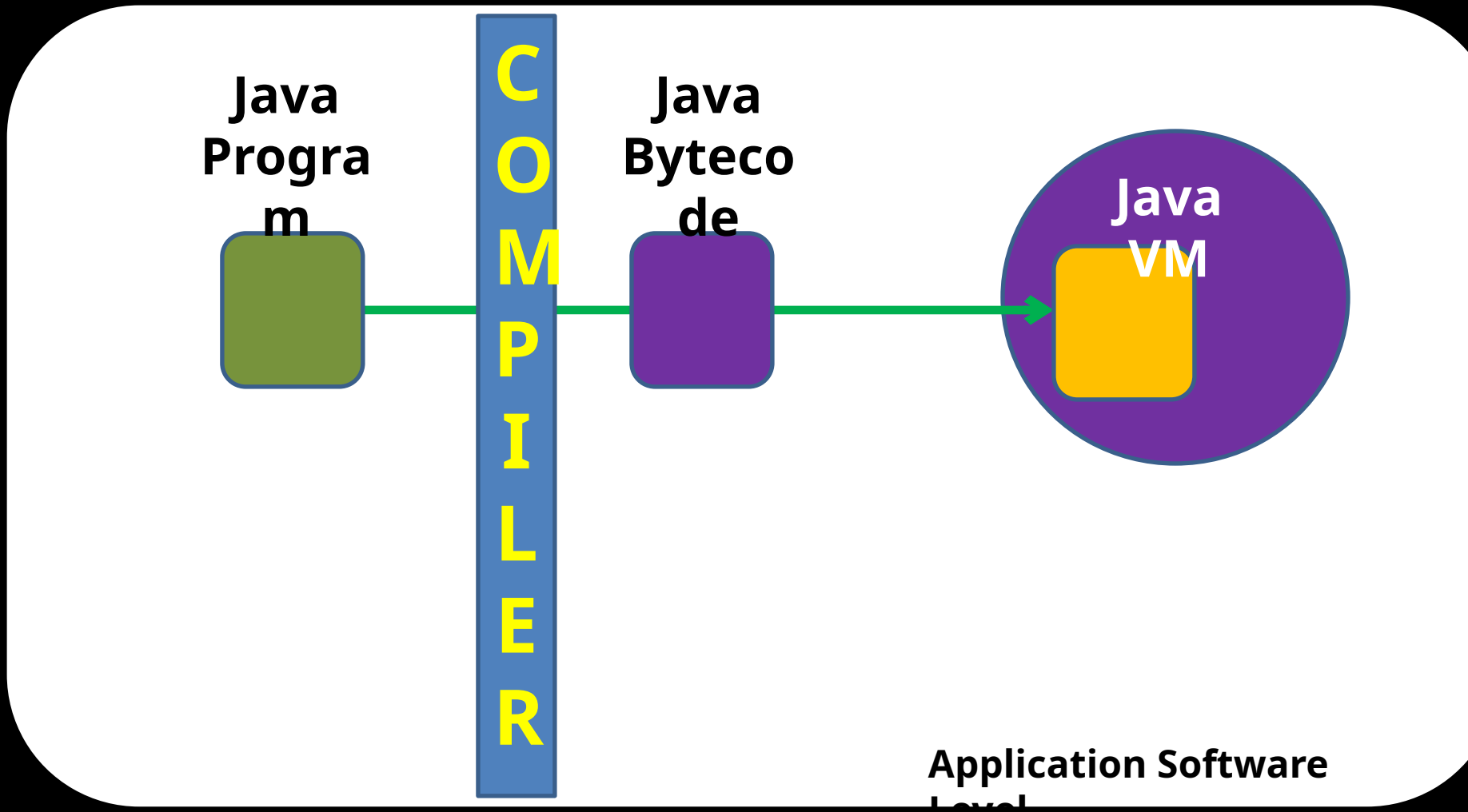
Source: http://cs.sru.edu/~mullins/cpsc100book/images/softwareLayers.jpg

WHAT ABOUT JAVA?

Virtual Machine

- Java is run on a "Virtual Machine."
- A Virtual Machine (VM) is an emulation of all of the components (hardware or software) necessary to run a certain piece of software.
- It is technically an application.

Java Flowchart



VM Pros and Cons

Pros:

- Code and bytecode works independently from the platform (operating system) it was compiled on.
- More secure due to runtime isolation.

Cons:

- Generally slower runtime (than natively compiled languages like C++).
- Arguably provides less fine-tuned usage of lower levels of software

My First Program

COMPILING AND COMMENTING

Hello World!

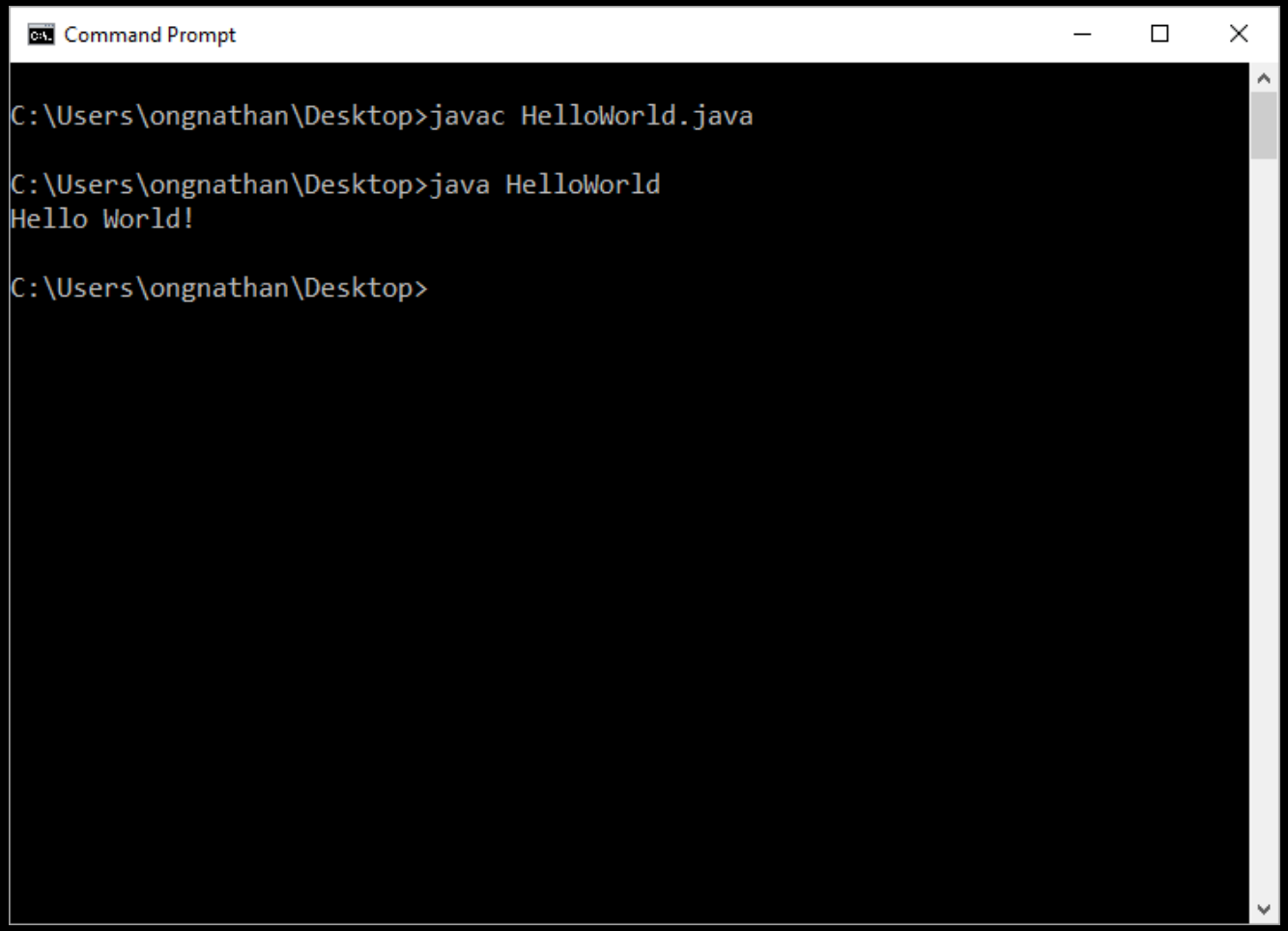
- A "Hello World" program is frequently a student's first program of any language.
- Similar to learning "Hello" greetings in other languages.

- Note: all code is CaSe SeNsItIvE

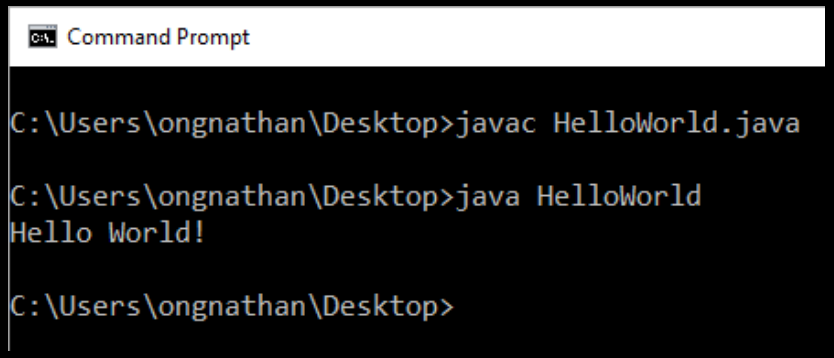
Hello World!

```
public class HelloWorld
{
	public static void main(String[] args)
	{
		System.out.println("Hello World!");
	}//end method main
}//End class HelloWorld
```

Compilation



Compiling



javac – compiling a java file

(javac FileName.java)

java – running a compiled java file

(java FileName)

Hello World!

```
public class HelloWorld
{
      public static void main(String[] args)
      {
            System.out.println("Hello World!");
      }//end method main
}//End class HelloWorld
```

```
System.out.println("Hello World!");
```

- Statement
- Ends with a semicolon (like a period in English)

- Prints stuff
  - Where to?
  - What stuff?
- To the console
- Strings

Hello World v2

```
public class Hello
{
	public static void main(String[] args)
	{
		System.out.println("My name is
			Nathan Ong! YAAAAYYYY LALALALA
			RANDOM TYPING");
	}//end method main
}//End class Hello
```

What is the output?

String

- Characters in between quotation marks
- Some valid Strings
  - `"Hi"`
  - `"#&^@!*(QQQQQQQQ"`
- What if we want quotation marks in the String?

\

- Escape character \ tells the computer that the next character should not be taken literally
  - \"
  - Tab? \t
  - New line? \n
  - Slash? \\
- `"Java said, "OMG SYNTAX ERROR""`
- How do we make the line above a valid String?

Hello World v2

```
public class Hello
{
	public static void main(String[] args)
	{
		System.out.println("My name is
			Nathan Ong! YAAAAYYYY LALALALA
			RANDOM TYPING");
	}//end method main
}//End class Hello
```

What if I want to use my name in another print statement?

Recycling

- What if I want to use the string somewhere else?
- I'm so lazy, I don't want to type in my name when printing it out all the time.
- `String name = "Nathan Ong";`
- Now I can call name wherever I want (not really)!

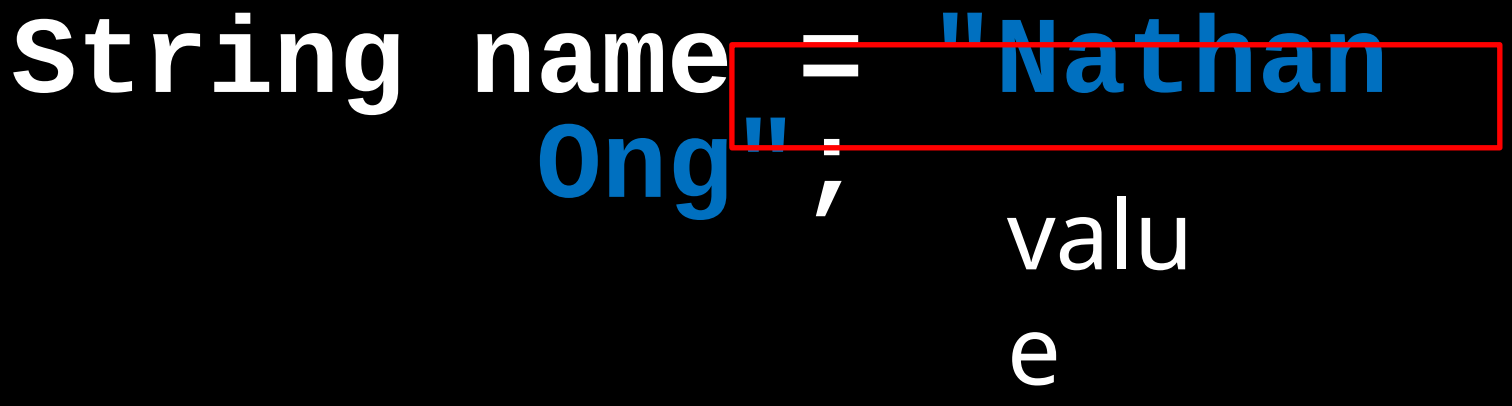
-->Variables<--

```
String name = "Nathan
              Ong";
```

```
String name = "Nathan Ong";
```

Type

```
String name = "Nathan
              Ong";
```


String name = "Nathan Ong";
valu
e

To Declare a Variable:

Type name = value;

To Use a Variable:

name

Naming

- The name of a variable (and other things) are called identifiers.
- There are some rules regarding valid identifiers, and further conventions that you should follow.

Valid Identifiers

- You must have at least one character in the name
- You can only use alphanumeric characters (letters and numbers), underscores (_), and dollar signs ($) in the name
- You CANNOT use a number as the first character.
- You CANNOT use reserved Java keywords.

Reserved Java Keywords

```
boolean   do  int    static
break double  long   super
byte  else    new        switch
case  final   package    this
catch finally private    throw
char  float   protected  throws
class for     public try
continue if   return void
default   import  short  while
```

Conventional Naming

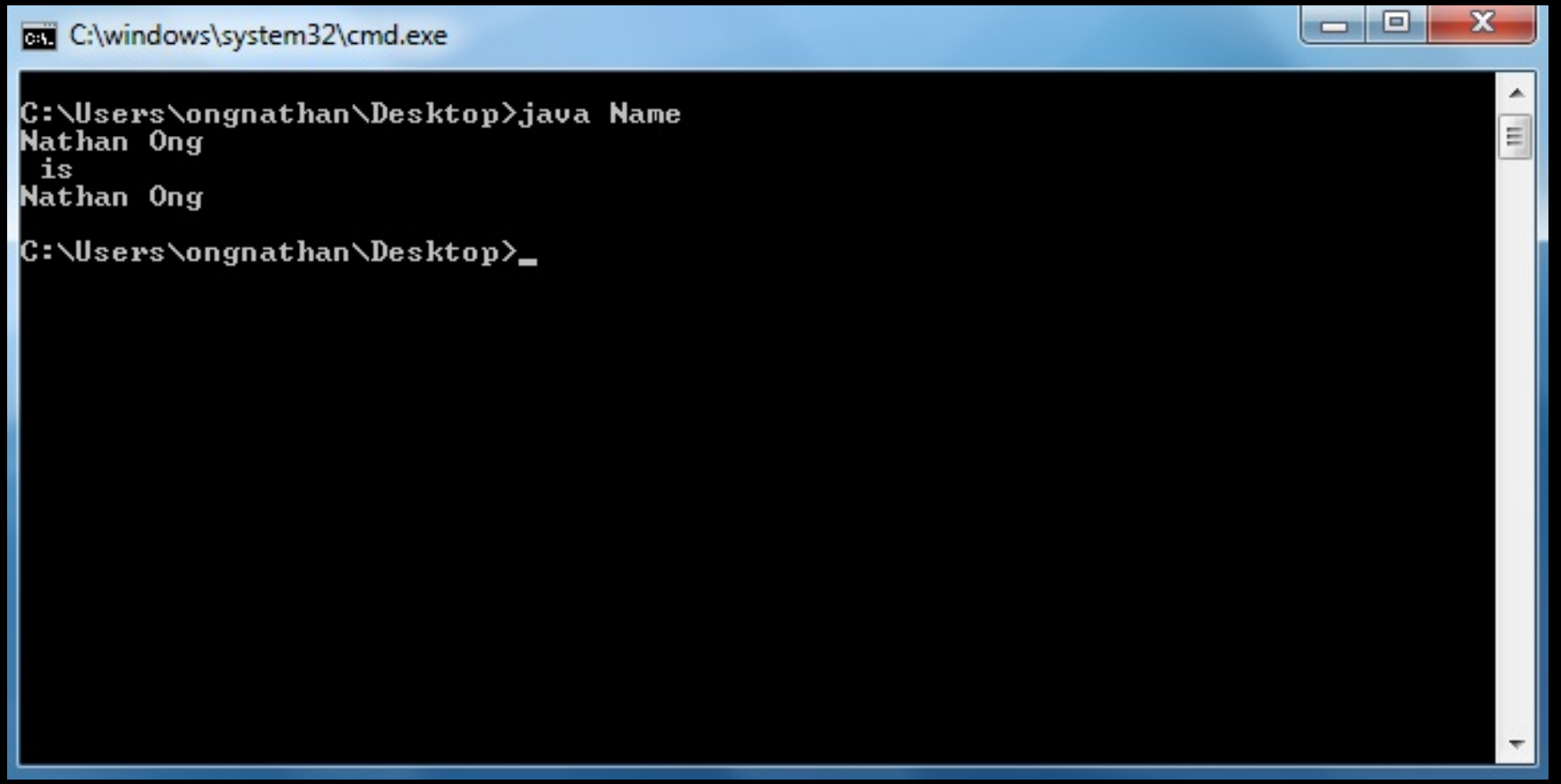
- Variables and functions: lowerCamelCase
  - First word starts with a lowercase letter, subsequent words have uppercase first letters
- Class names: UpperCamelCase
  - All words have uppercase first letters

- Never use dollar signs or underscores

Hello World v3

```
public class Name
{
        public static void main(String[] args)
        {
                String name = "Nathan Ong";
                System.out.println(name);
                System.out.println(" is ");
                System.out.println(name);
      }//end method main
}//End class Name
```

What is the output?



Oh I changed my name...Nathan Ong is no longer Nathan Ong...What am I going to do?

What can we do with it?

```
public class Name
{
  public static void main(String[] args)
  {
  String name = "Nathan Ong";
  System.out.println(name);
  System.out.println(" is ");
  String name = "Brandon Ong";
  System.out.println(name);
      }//end method main
}//End class Name
```

Can we do this?

No, because we re-declared a variable!

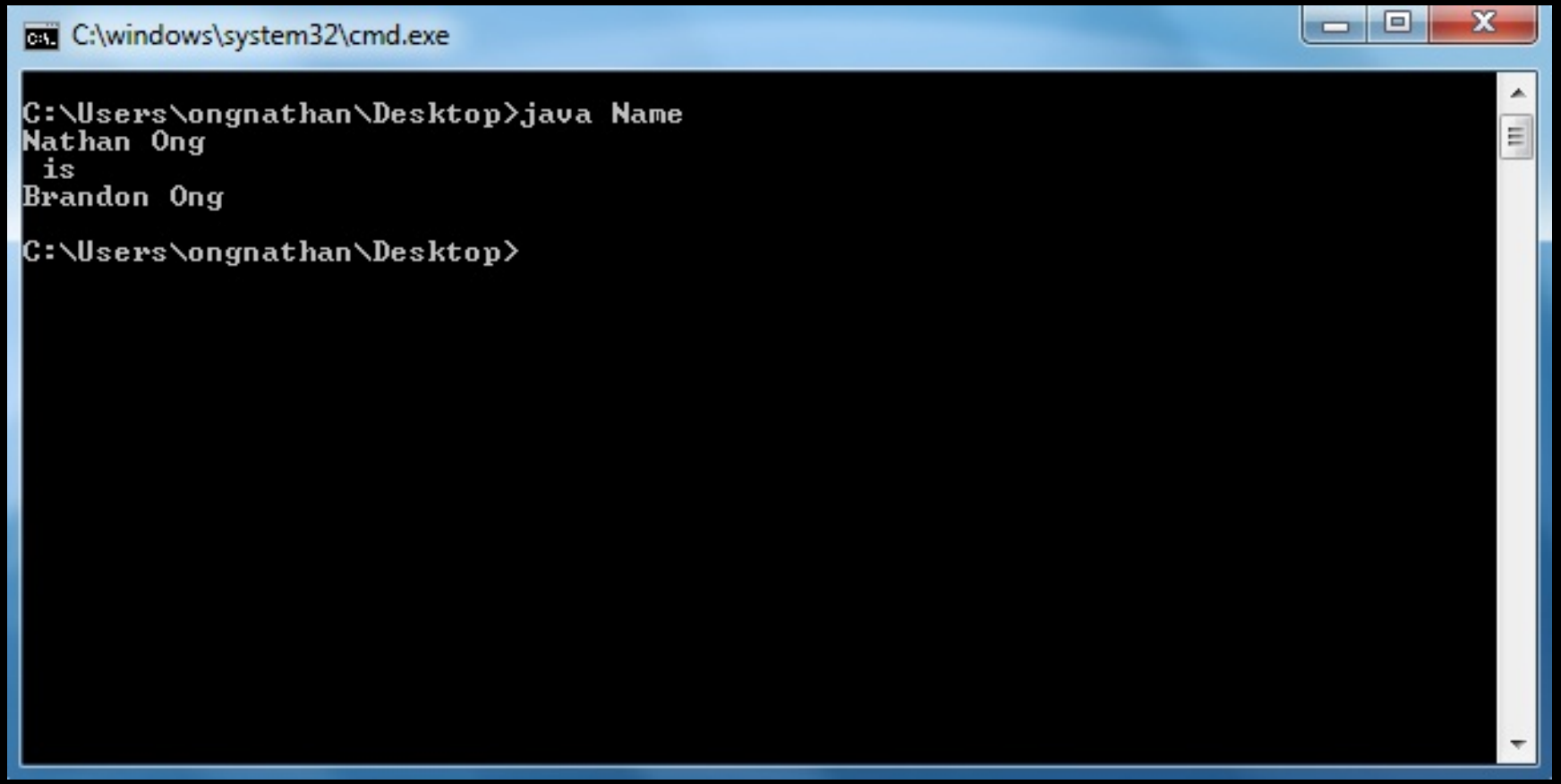
Declaring a new variable:
Type name = value;

Changing the old variable:
name = newValue;

What can we do with it?

```
public class Name
{
        public static void main(String[] args)
        {
        String name = "Nathan Ong";
        System.out.println(name);
        System.out.println(" is ");
        name = "Brandon Ong";
        System.out.println(name);
      }//end method main
}//End class Name
```

What is the output?



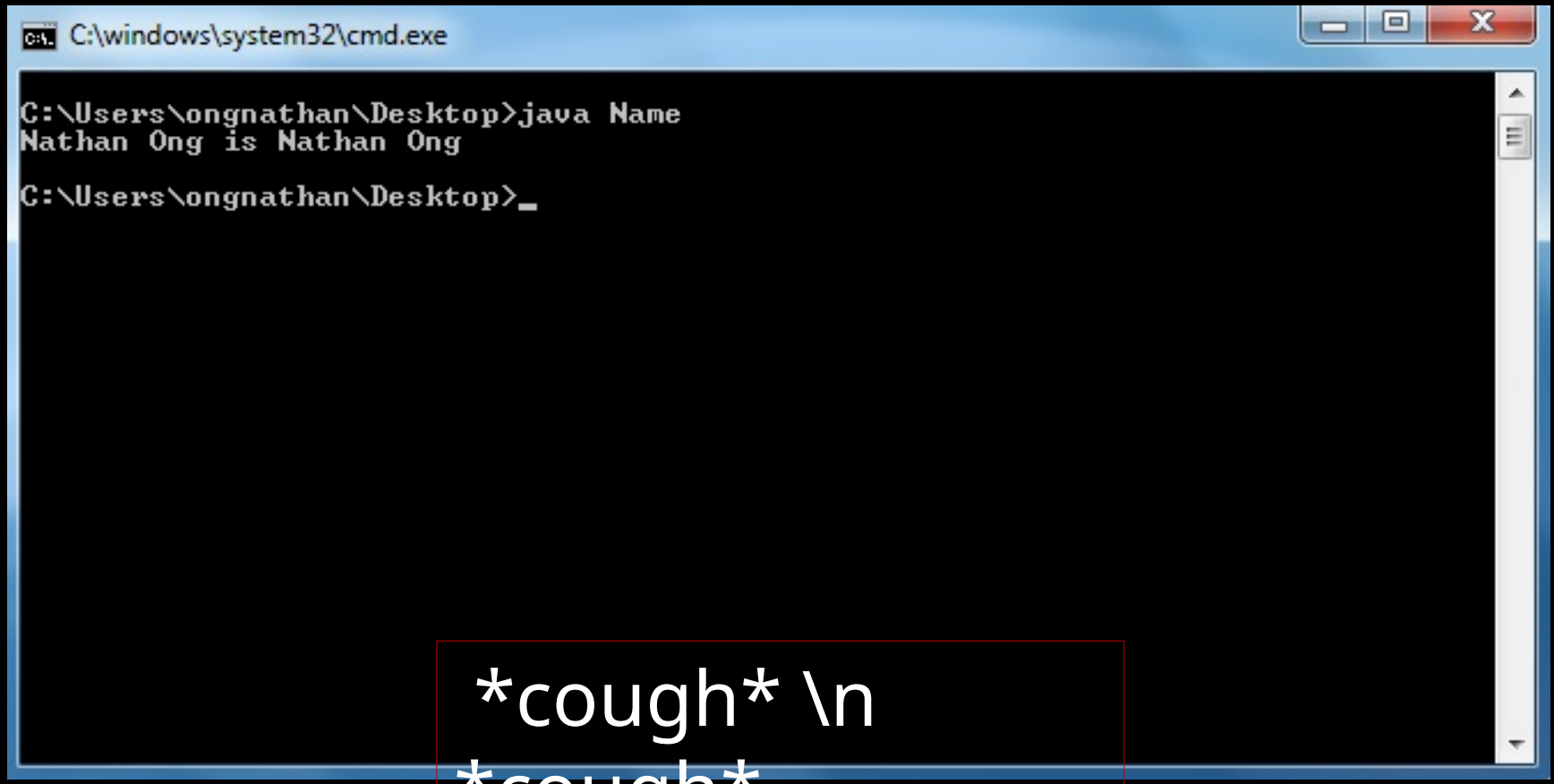
Okay so I'm not really changing my name.
Regardless, the output is really annoying.
How do we put it all in one line?

What can we do with it?

```
public class Name
{
        public static void main(String[] args)
        {
        String name = "Nathan Ong";
        System.out.print(name);
        System.out.print(" is ");
        System.out.println(name);
      }//end method main
}//End class Name
```

Use print instead. No new line added.

Saying print print print is kind of annoying...



cough \n
cough

Ahhhhh, that's better.

But just for fun, how do we get it back to the multiple-line version using the print statements instead of println?

What can we do with it?

```
public class Name
{
  public static void main(String[] args)
  {
  String name = "Nathan Ong";
  System.out.print(name);
  System.out.print("\n is \n");
  System.out.println(name);
     }//end method main
}//End class Name
```

Getting the same output from before with only print. But we don't like that, so we'll take that out

Saying print print print is kind of annoying...

What can we do with it?

```
public class Name
{
   public static void main(String[] args)
   {
   String name = "Nathan Ong";
   System.out.println(name + " is " + name);
   }//end method main
}//End class Name
```

Using + with Strings is called concatenation

(Note: there's no such thing as – for Strings)

What can we do with it?

```
public class Name
{
        public static void main(String[] args)
        {
                String name = "Nathan Ong";
                System.out.println(name + " is " +
  name);
        }//end method main
}//End class Name
```

Concatenation does not add spaces automatically.

Nicknames

- `System.out.println(name + " is " + name);`
- Seems useful, like if someone had a nickname
- `System.out.println(nickName + " is " + name);`
- But I don't have one...how do we make it so it doesn't matter?

Nicknames

```
public class Name
{
        public static void main(String[] args)
        {
                String name = "Nathan Ong";
                String nickName = name;
                System.out.println(nickName + " is " +
   name);
        }//end method main
}//End class Name
```

Oh so we can make nickName the exact same thing as name without having to type everything again!

What is the output?


C:\windows\system32\cmd.exe
C:\Users\ongnathan\Desktop>java Name
Nathan Ong is Nathan Ong
C:\Users\ongnathan\Desktop>

Same thing

Comments

```
public class Name
{
        public static void main(String[] args)
        {
                String name = "Nathan Ong";
                String nickName = name;
                System.out.println(nickName + " is " +
                        name);
        }//end method main
}//End class Name
```

Commenting

- You must comment every line of useful code.
- Don't forget the //
- If you need a block of comments, use /* */
- Everything in between is commented out
- Careful about nested comments...
- /* This /* Comment Ends */ Here???*/

Line-by-Line Commenting

```
//The file name
^Bad comment
//Prints out statistics for
//Nathan Ong
^Good Comment
public class Name
```

Line-by-Line Commenting

```
//The main method
^Bad comment
//The program begins here
^Good Comment
public static void main(String[]
  args)
```

Line-by-Line Commenting

```
//A String
^Bad comment
//The first name held in a
//String
^Good Comment
String name = "Nathan";
```

Line-by-Line Commenting

```
//Prints out stuff
```

^Bad comment

```
//Prints out the names
```

^Okay comment

```
/*Prints the nickname and links it to
  the original name.*/
```

^Good comment

```
System.out.println(nickName +
          " is " + name);
```

Recap

- Hello World!
- System.out.println (and print)
- Strings and String concatenation
- Compiling and running
- Variables

```
Type name = value;
name = newValue;
```

- Commenting